

Robustifying Profile Information Propagation in Profile-Guided Optimization



SAPIENZA
UNIVERSITÀ DI ROMA

Table of Contents

1. Profile-Guided Optimization
2. Practical Obstacles
3. State of the Art
4. Profile Mishandling Research Gap
5. Research Proposal
6. Evaluation of the Proposed Directions
7. Impacts and Benefits

Performance is Critical

Performance is Critical

- Achieving optimal performance is of critical importance today.
 - A 1% performance improvement can save millions of dollars annually in large-scale deployments.

Performance is Critical

- Achieving optimal performance is of critical importance today.
 - A 1% performance improvement can save millions of dollars annually in large-scale deployments.
- **Profile-Guided Optimization** is a way to achieve great performance
 - Optimization process **tailored** to the workload of the compiled program
 - **Profile** = Execution frequencies of program regions
 - Eliminates the need to infer such information
 - Perfect for high-load applications

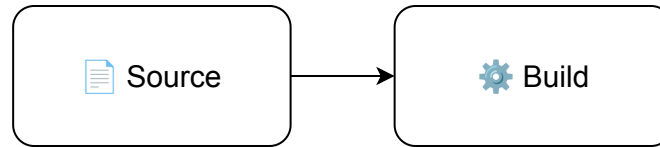
Profile-Guided Optimization

Profile-Guided Optimization

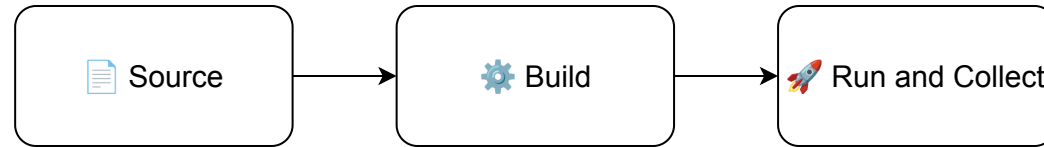


Source

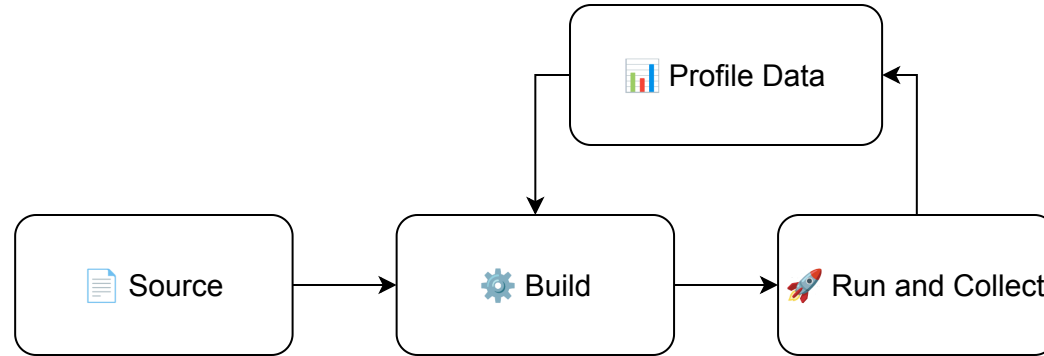
Profile-Guided Optimization



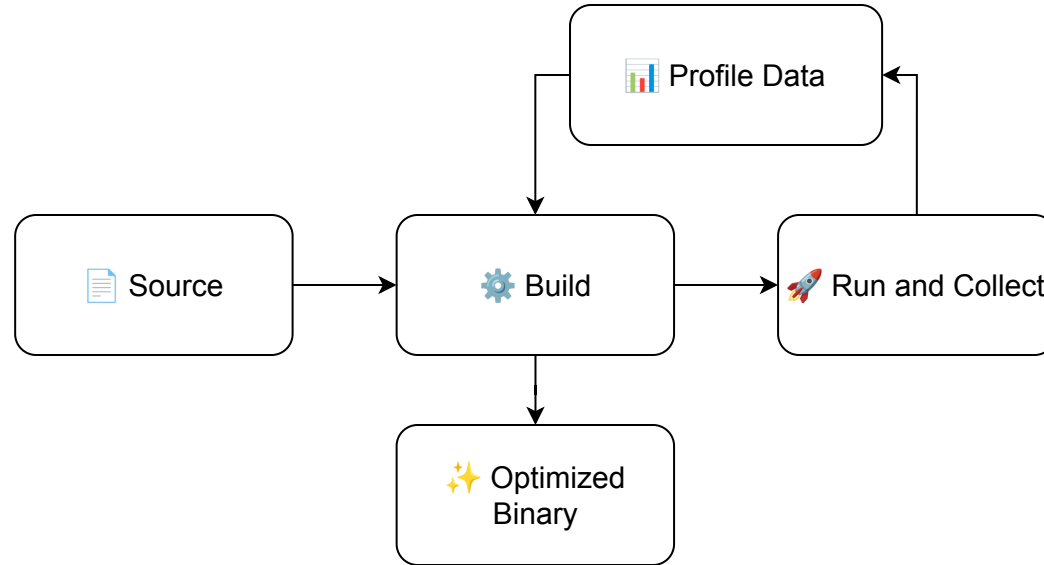
Profile-Guided Optimization



Profile-Guided Optimization



Profile-Guided Optimization



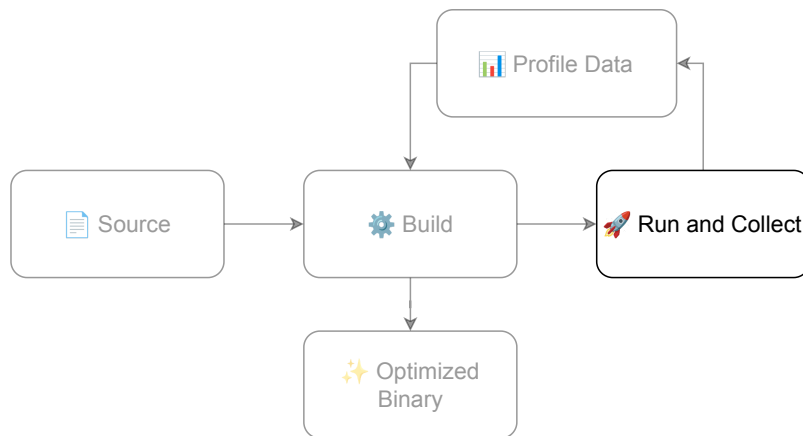
Profile Life-cycle

Profile Life-cycle

- Profiles traverse three main phases within the PGO workflow

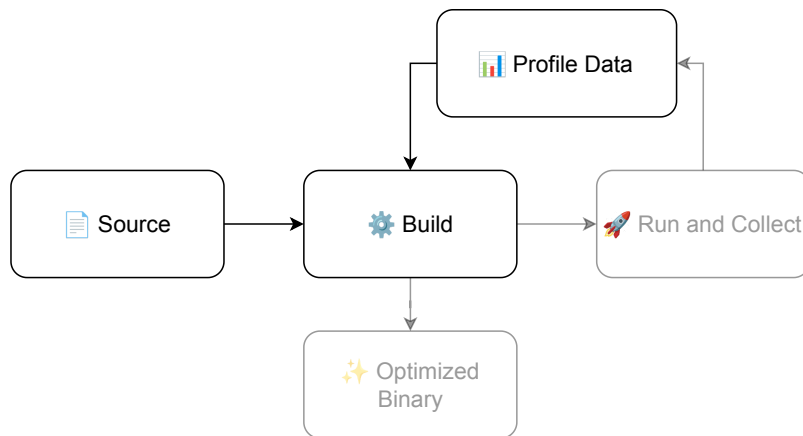
Profile Life-cycle

- Profiles traverse three main phases within the PGO workflow
 - **Collection:** The profile is collected and persisted as a file



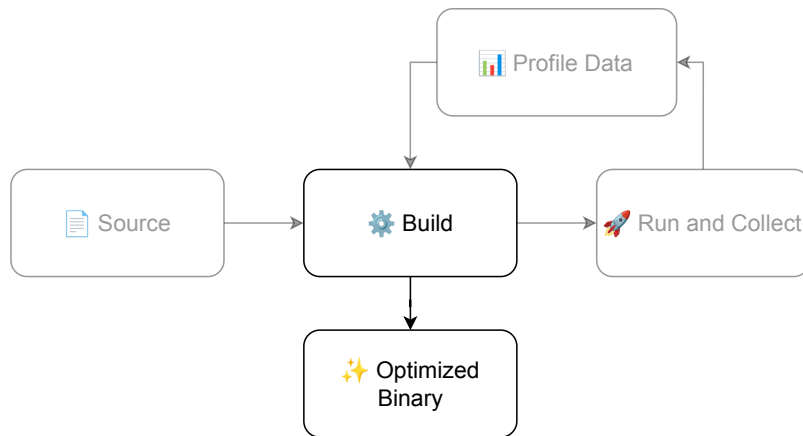
Profile Life-cycle

- Profiles traverse three main phases within the PGO workflow
 - **Collection:** The profile is collected and persisted as a file
 - **Mapping:** The profile is represented as **metadata** attached to program constructs



Profile Life-cycle

- Profiles traverse three main phases within the PGO workflow
 - **Collection:** The profile is collected and persisted as a file
 - **Mapping:** The profile is represented as **metadata** attached to program constructs
 - **Usage:** The metadata is used to guide optimizing transformations



Practical Obstacles

- For an ideal PGO application



Profile must be **complete** and **accurate** in each phase of its life-cycle

Practical Obstacles

- For an ideal PGO application



Profile must be **complete** and **accurate** in each phase of its life-cycle

- Inaccurate profiles may lead the compiler to make suboptimal optimization decisions.

Practical Obstacles

- For an ideal PGO application



Profile must be **complete** and **accurate** in each phase of its life-cycle

- Inaccurate profiles may lead the compiler to make suboptimal optimization decisions.
- In practice, each phase hides sources of inaccuracies
 - **Sampling** strategies are inaccurate by nature
 - The dynamic nature of software leads to **stale profiles**
 - Optimizing passes contain errors in **metadata propagation** logic

State of the Art

1. Wenlei He, Julián Mestre, Sergey Pupyrev, Lei Wang, and Hongtao Yu. “Profile inference revisited”. ↵
2. Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. “Stale Profile Matching”. ↵
3. Youfeng Wu. “Accuracy of Profile Maintenance in Optimizing Compilers”. ↵
4. Profile Information Propagation Unittesting: <https://discourse.llvm.org/t/rfc-profile-information-propagation-unittesting/73595> ↵

State of the Art

- Lots of effort to solve problems in the collection and mapping phases

1. Wenlei He, Julián Mestre, Sergey Pupyrev, Lei Wang, and Hongtao Yu. "Profile inference revisited". ↵
2. Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. "Stale Profile Matching". ↵
3. Youfeng Wu. "Accuracy of Profile Maintenance in Optimizing Compilers". ↵
4. Profile Information Propagation Unittesting: <https://discourse.llvm.org/t/rfc-profile-information-propagation-unittesting/73595> ↵

State of the Art

- Lots of effort to solve problems in the collection and mapping phases
 - [1] Proposes a rectification algorithm to rectify sampled profiles

1. Wenlei He, Julián Mestre, Sergey Pupyrev, Lei Wang, and Hongtao Yu. "Profile inference revisited". ↵
2. Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. "Stale Profile Matching". ↵
3. Youfeng Wu. "Accuracy of Profile Maintenance in Optimizing Compilers". ↵
4. Profile Information Propagation Unittesting: <https://discourse.llvm.org/t/rfc-profile-information-propagation-unittesting/73595> ↵

State of the Art

- Lots of effort to solve problems in the collection and mapping phases
 - ^[1] Proposes a rectification algorithm to rectify sampled profiles
 - ^[2] Proposes an algorithm to adapt stale profiles to newer program versions

1. Wenlei He, Julián Mestre, Sergey Pupyrev, Lei Wang, and Hongtao Yu. “Profile inference revisited”. ↵
2. Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. “Stale Profile Matching”. ↵
3. Youfeng Wu. “Accuracy of Profile Maintenance in Optimizing Compilers”. ↵
4. Profile Information Propagation Unittesting: <https://discourse.llvm.org/t/rfc-profile-information-propagation-unittesting/73595> ↵

State of the Art

- Lots of effort to solve problems in the collection and mapping phases
 - [\[1\]](#) Proposes a rectification algorithm to rectify sampled profiles
 - [\[2\]](#) Proposes an algorithm to adapt stale profiles to newer program versions
 - [\[3\]](#), [\[4\]](#) only partially address failures in metadata propagation

1. Wenlei He, Julián Mestre, Sergey Pupyrev, Lei Wang, and Hongtao Yu. “Profile inference revisited”. ↵

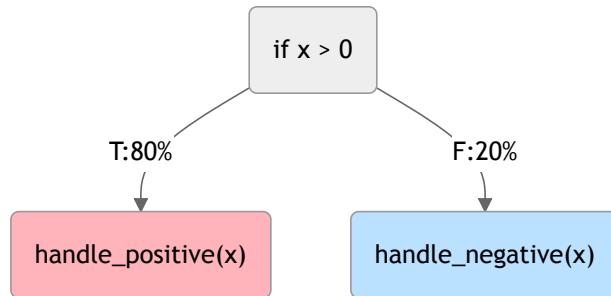
2. Amir Ayupov, Maksim Panchenko, and Sergey Pupyrev. “Stale Profile Matching”. ↵

3. Youfeng Wu. “Accuracy of Profile Maintenance in Optimizing Compilers”. ↵

4. Profile Information Propagation Unittesting: <https://discourse.llvm.org/t/rfc-profile-information-propagation-unittesting/73595> ↵

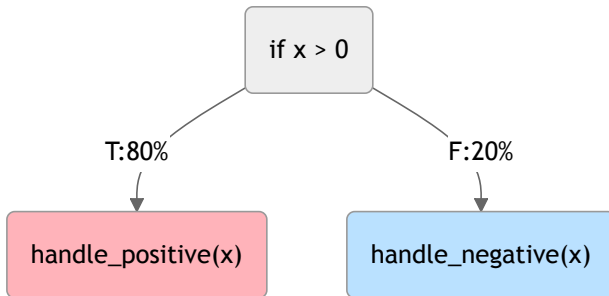
Example of Profile Mishandling

```
// Before pass  
if (x > 0) { // Then branch taken 80 times  
    handle_positive(x) 🔥  
} else { // Else branch taken 20 times  
    handle_negative(x) ❄️  
}
```

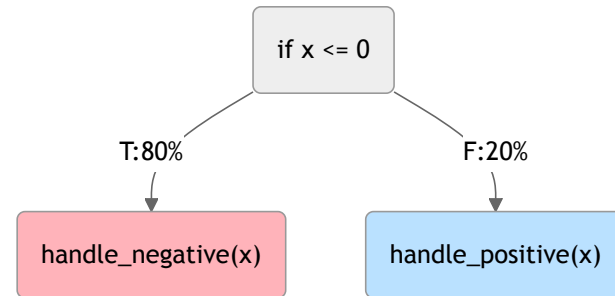


Example of Profile Mishandling

```
// Before pass
if (x > 0) { // Then branch taken 80 times
    handle_positive(x) 🔥
} else { // Else branch taken 20 times
    handle_negative(x) ❄️
}
```



```
// After pass
if (x ≤ 0) { // Then branch taken 80 times
    handle_negative(x) 🔥
} else { // Else branch taken 20 times
    handle_positive(x) ❄️
}
```



Profile Mishandling Research Gap

Profile Mishandling Research Gap

- Recently discovered **regressions** could be attributed to profile mishandling

Profile Mishandling Research Gap

- Recently discovered **regressions** could be attributed to profile mishandling
- Profile mishandling **nullifies** solutions aiming at correcting profiles in earlier stages

Profile Mishandling Research Gap

- Recently discovered **regressions** could be attributed to profile mishandling
- Profile mishandling **nullifies** solutions aiming at correcting profiles in earlier stages
- **Non-trivial** solution
 - What does it mean to correctly propagate profiles?
 - No immediate correlation between before and after code
 - **Interaction** between passes needs to be taken into account
 - **Static checks** on control-flow are not enough!
 - **No dedicated tools** to validate propagation logic

Profile Mishandling Research Gap

- Recently discovered **regressions** could be attributed to profile mishandling
- Profile mishandling **nullifies** solutions aiming at correcting profiles in earlier stages
- **Non-trivial** solution
 - What does it mean to correctly propagate profiles?
 - No immediate correlation between before and after code
 - **Interaction** between passes needs to be taken into account
 - **Static checks** on control-flow are not enough!
 - **No dedicated tools** to validate propagation logic



Profile information is **transformed** together with the program, but unlike the program itself, its correctness cannot be directly observed.

Research Proposal



Can profile propagation accuracy be assessed systematically?

Research Proposal



Can profile propagation accuracy be assessed systematically?

- The definition of profile **validation** policies
 - A systematic way to spot **incorrect** profiles

Research Proposal

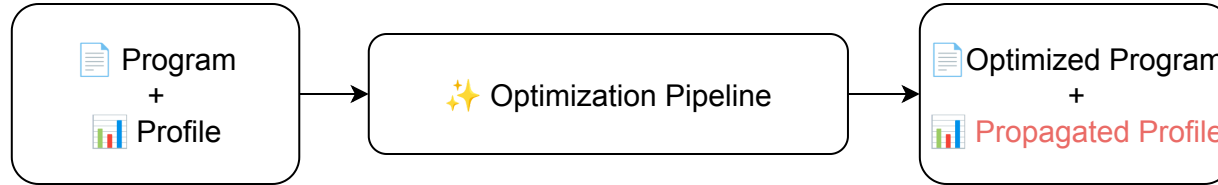


Can profile propagation accuracy be assessed systematically?

- The definition of profile **validation** policies
 - A systematic way to spot **incorrect** profiles
- A methodological and practical framework to
 - Assess the **soundness** of validation policies
 - Stress-test the compiler in order to **expose** profile propagation errors
 - Accurately **pinpoint** causes of inaccuracies

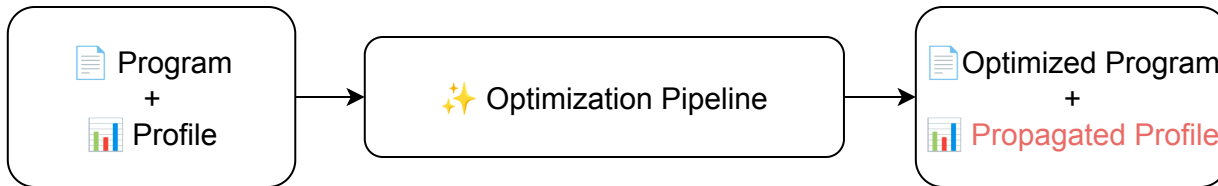
Validation Policies

- What does it mean for a profile to be accurately propagated?



Validation Policies

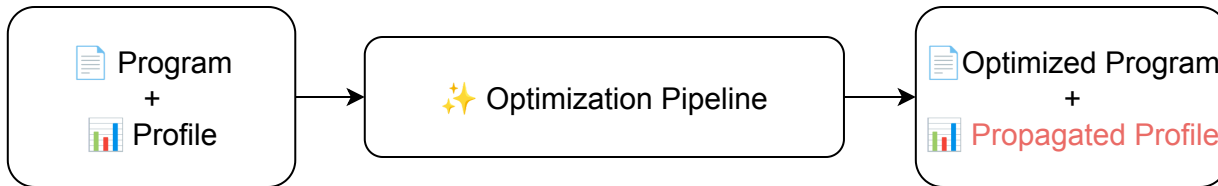
- What does it mean for a profile to be accurately propagated?



- Checking if the profile dictates a **flow that makes sense**
 - Example: checking if the profile follows **flow-conservation rules**
 - Good starting point, but not enough

Validation Policies

- What does it mean for a profile to be accurately propagated?



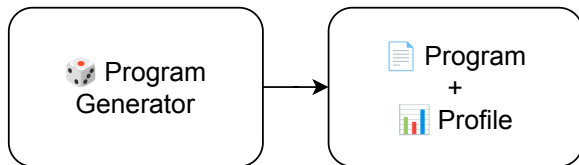
- Checking if the profile dictates a **flow that makes sense**
 - Example: checking if the profile follows **flow-conservation rules**
 - Good starting point, but not enough
- Checking the profile against **ground-truth** values
 - Computed by **re-collecting** a profile for the optimized program
 - Compare** the compiler generated profile against ground-truth values

Random Programs Approach

- **Randomly** generated programs
 - Uses **off-the-shelf** random program generators
 - Exploit **complexity** of generated programs to uncover superficial bugs
 - Automated **bug triaging** to classify the large number of issues

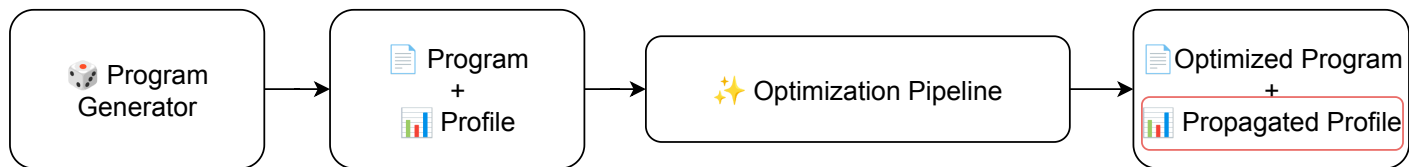
Random Programs Approach

- **Randomly** generated programs
 - Uses **off-the-shelf** random program generators
 - Exploit **complexity** of generated programs to uncover superficial bugs
 - Automated **bug triaging** to classify the large number of issues



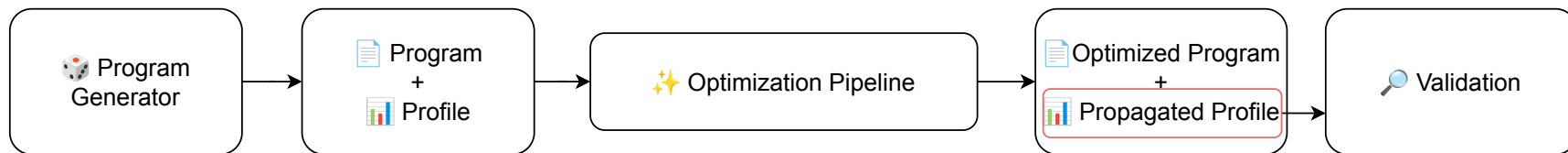
Random Programs Approach

- **Randomly** generated programs
 - Uses **off-the-shelf** random program generators
 - Exploit **complexity** of generated programs to uncover superficial bugs
 - Automated **bug triaging** to classify the large number of issues



Random Programs Approach

- **Randomly** generated programs
 - Uses **off-the-shelf** random program generators
 - Exploit **complexity** of generated programs to uncover superficial bugs
 - Automated **bug triaging** to classify the large number of issues



Mutations Approach

- **Mutations** of existing test suite programs
 - **Feedback** mechanism through the instrumentation of optimization passes
 - Code and **profile** mutations guided by **custom coverage** metric
 - Exposes **untested** regions of the compiler to uncover deeper issues

Mutations Approach

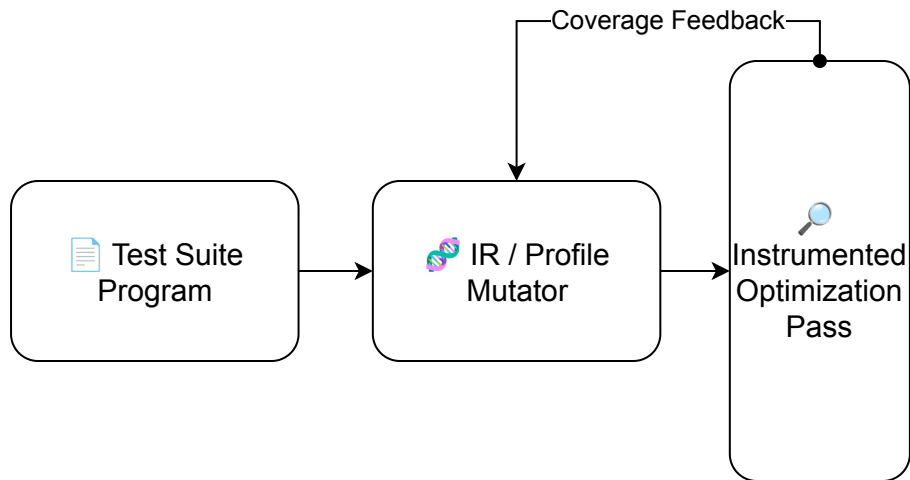
- **Mutations** of existing test suite programs
 - **Feedback** mechanism through the instrumentation of optimization passes
 - Code and **profile** mutations guided by **custom coverage** metric
 - Exposes **untested** regions of the compiler to uncover deeper issues



Test Suite
Program

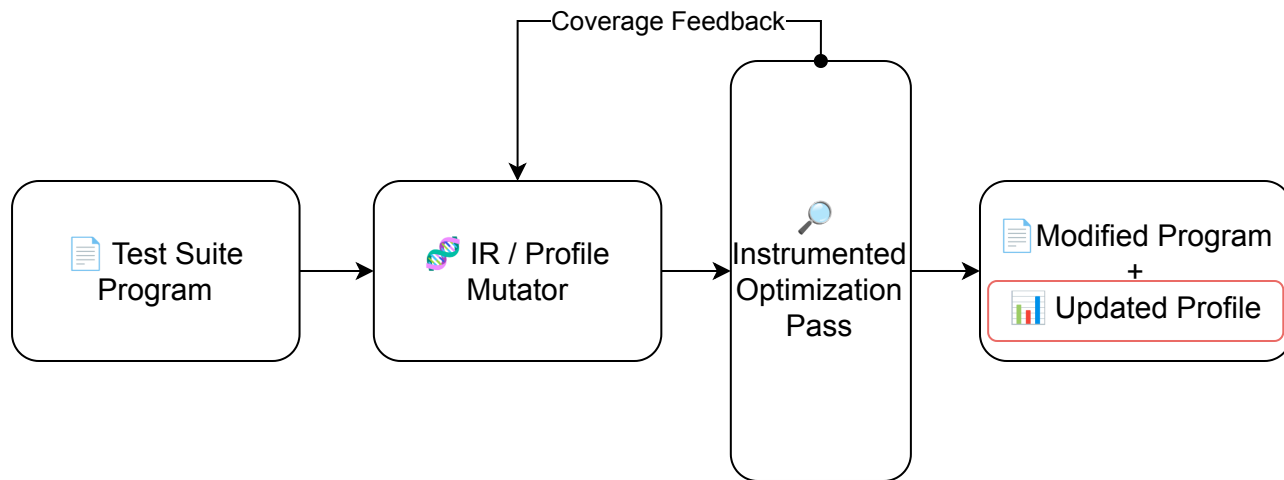
Mutations Approach

- **Mutations** of existing test suite programs
 - **Feedback** mechanism through the instrumentation of optimization passes
 - Code and **profile** mutations guided by **custom coverage** metric
 - Exposes **untested** regions of the compiler to uncover deeper issues



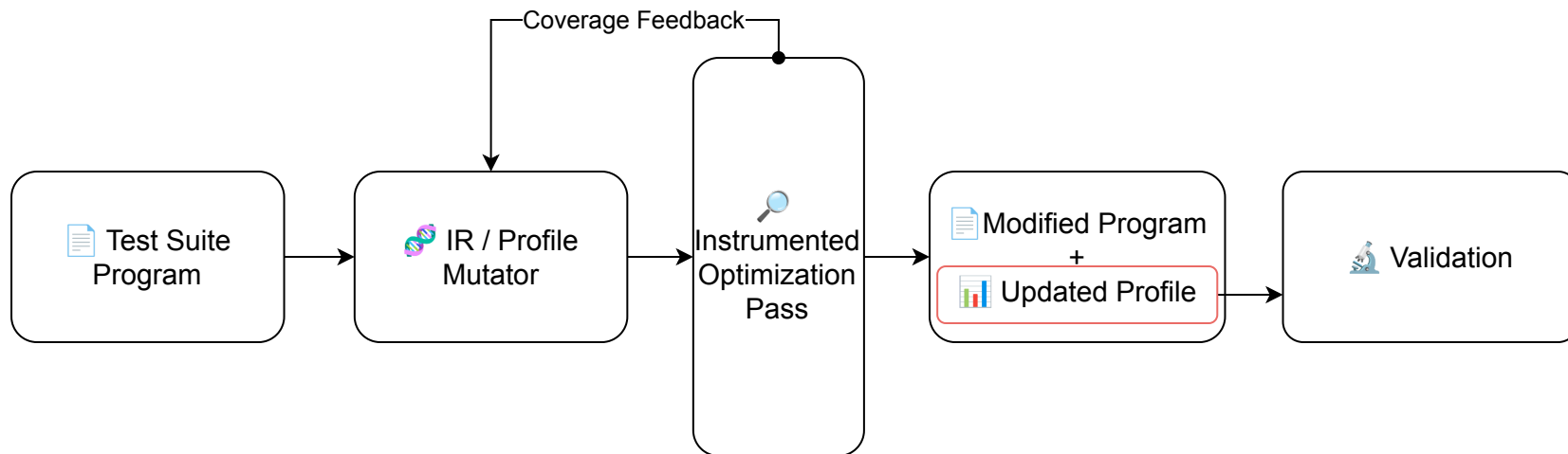
Mutations Approach

- **Mutations** of existing test suite programs
 - **Feedback** mechanism through the instrumentation of optimization passes
 - Code and **profile** mutations guided by **custom coverage** metric
 - Exposes **untested** regions of the compiler to uncover deeper issues



Mutations Approach

- **Mutations** of existing test suite programs
 - **Feedback** mechanism through the instrumentation of optimization passes
 - Code and **profile** mutations guided by **custom coverage** metric
 - Exposes **untested** regions of the compiler to uncover deeper issues



Evaluation of the Proposed Directions

Evaluation of the Proposed Directions

- **LLVM** compiler infrastructure as evaluation target

Evaluation of the Proposed Directions

- **LLVM** compiler infrastructure as evaluation target
- Can the proposed methodologies:

Evaluation of the Proposed Directions

- **LLVM** compiler infrastructure as evaluation target
- Can the proposed methodologies:
 - Find new **bugs**? If so, how many?

 Bugs

Evaluation of the Proposed Directions

- **LLVM** compiler infrastructure as evaluation target
- Can the proposed methodologies:
 - Find new **bugs**? If so, how many?
 - **Improve the performance** of the generated binaries?



Bugs



Performance

Evaluation of the Proposed Directions

- **LLVM** compiler infrastructure as evaluation target
- Can the proposed methodologies:
 - Find new **bugs**? If so, how many?
 - **Improve the performance** of the generated binaries?
 - Improve optimization pass coverage within LLVM?



Bugs



Performance



Coverage

Impacts and Benefits

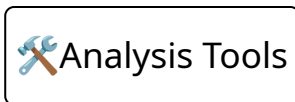
Impacts and Benefits

- **Analysis tools** for compiler developers, to assess their PGO implementations



Impacts and Benefits

- **Analysis tools** for compiler developers, to assess their PGO implementations
- **Faster** PGO binaries for the user



Impacts and Benefits

- **Analysis tools** for compiler developers, to assess their PGO implementations
- **Faster** PGO binaries for the user
- **Energy savings** due to optimal binaries



Analysis Tools



Faster Binaries



Energy Savings

Thanks for the attention!
Questions?